

На вкус и цвет все фломастеры разные

Особенно если вы пишете на Ruby

Павел Аргентов · RubyRussia 2026

Как я провёл...

1. Что?
2. Ай дид май овн рисёрч
3. Эксперимент
4. Гладко было на бумаге...
5. Первые наблюдения
6. Обзор на обзор на эксперимент
7. В заключение

Что?

Условия работы

- Аутстафф. Клиенты меняются
- Обычная задача — доработать или переработать чужой код (писал человек или машина)
- Язык может быть любым

“

*Машина пишет код, а человек
получает...*

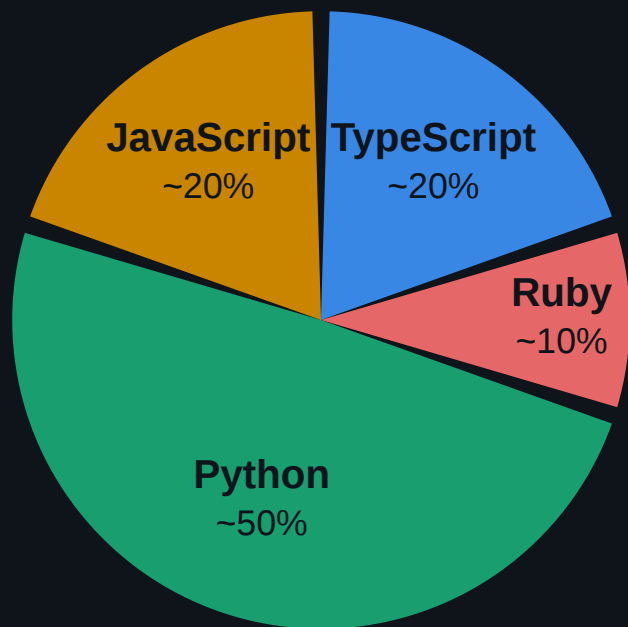
Типичный сценарий

- Задача — доработать или переработать (иногда и с нуля написать)
- Код чужой, на произвольном языке
- Нужно на ходу оценить, насколько AI-агент «вывозит» этот язык
- Если модель плохо справляется, приходится компенсировать руками

Ай дид май овн рисёрч

Гипотеза

Агент пристрастен к языку из-за объёма его кода в обучающей выборке модели.



■ **Python** — язык авторов LLM

■ **JavaScript** — массовость

■ **TypeScript** — тот же JS

■ **Ruby** — МЁРТВ

Эксперимент

Подопытные

Python

JavaScript

TypeScript

Ruby

Харнесс

Claude Code, headless (`claude`
`-p`)

Модель

`claude-sonnet-5` , точным ID на
каждом вызове

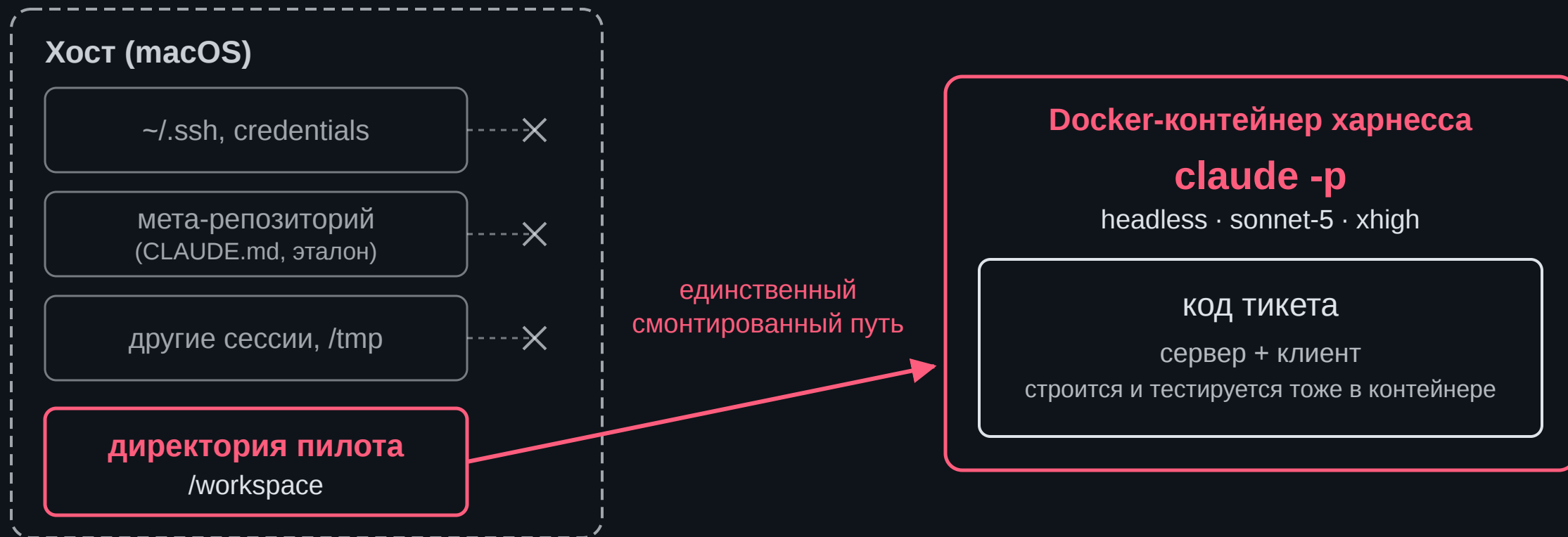
Effort

`xhigh` , явно на каждом вызове

Переменная

только язык проекта — всё
остальное держим
постоянным

Установка



- Изоляция агента на уровне ОС
- Если путь не примонтирован — его физически нет в файловой системе процесса агента

Процесс



- Один и тот же цикл на всех языках
- Три гейта — часть критерия «сошлось / упало»
- На тикет фиксируются **итерации**, **время** и **токены** — точными полями из JSON-вывода харнесса

Гейты

Гейт	Что проверяет	Режим
<code>run-tests</code>	штатные тесты (пишет проверяемый агент)	<code>block</code>
<code>contract-test.sh</code>	Schemathesis по OpenAPI	<code>block</code>
<code>smoke.sh</code>	чёрный ящик по HTTP/CLI	<code>info</code>

При падении блокирующего гейта харнесс сам строит «симптом» для следующего вызова `claude -p` — без путей хоста и имён acceptance-скриптов, до сходимости или `--max-iters` (по умолчанию 4).

Тестовая задача: Syncbox

- Файловое хранилище с синхронизацией
- HTTP-сервер + CLI-клиент
- Единая спецификация, с нуля на каждом языке

Жёсткие требования

- HTTP API: `/blobs/{key}`, `/blobs`, `/healthz`
- CLI: `push` / `pull` / `sync` / `status`, имена флагов
- Правило разрешения конфликтов в `sync`
- Тесты

На усмотрение «разработчика»

- Библиотеки, HTTP-фреймворк, тест-фреймворк
- Формат хранения метаданных на диске
- Модель параллелизма

Synsbox, ожидаемое поведение

```
$ run-server --data-dir ./data --port 8080
[слушает :8080]

$ run-client push ./notes --server http://localhost:8080
notes/todo.txt    201 Created    sha256=1a2b3c...

$ run-client sync ./notes --server http://localhost:8080
notes/plan.md     ← pulled (новее на сервере)
notes/todo.txt    up to date
```

- `run-server` / `run-client` — обязательная конвенция запуска
- `push` / `pull` — односторонне, отправка и получение
- `sync` — двунаправленно, конфликт решает более свежий `mtime`

Репозиторий



Репозиторий эксперимента на GitHub

[**github.com/argent-smith/llm-lang-experiment**](https://github.com/argent-smith/llm-lang-experiment)

Гладко было на бумаге...

Раунд 1: агент нашел эталонную реализацию

Обнаружено

- Тикет 1, Python — первый реальный прогон пилота
- Агент нашёл эталонную реализацию в мета-репозитории обычными Bash и Read
- Скопировал почти дословно — воспроизводилось на 100 %

Фикс

- `SYNCBOX-SPEC.md` больше не называет и не описывает пути внутри мета-репозитория
- Файловая песочница
`denyRead / allowRead` через `-- settings`

Раунд 2: агент снова все нашел

Обнаружено

- Аудит: агент опять все нашел и списал
- Песочница раунда 1 ограничивала только Bash-подпроцессы
- Инструменты Read и Write агента её не соблюдали вообще

Фикс

- Весь вызов `claude -p` обёрнут в Docker-контейнер
- С хоста в него примонтирована только директория пилота

Агент умный и находит любые обходные пути

Раунд 3: угадайте, что...

Обнаружено

- Тикет 9, Ruby
- Через сброшенный хостовый сокет Docker (DooD) агент мог примонтировать любой хостовый путь
- `run-client status /tmp` обошёл весь хостовый `/private/tmp`

Фикс

- Переход на Docker-in-Docker
- У внутреннего `dockerd` хостовой файловой системы в принципе нет

Агент «под пытками» сам во всём сознался и всё починил.

“

*Плохо сколоченный стенд
измеряет себя,
а не объект*

Первые наблюдения

Пилотная фаза — проверка состоятельности опыта

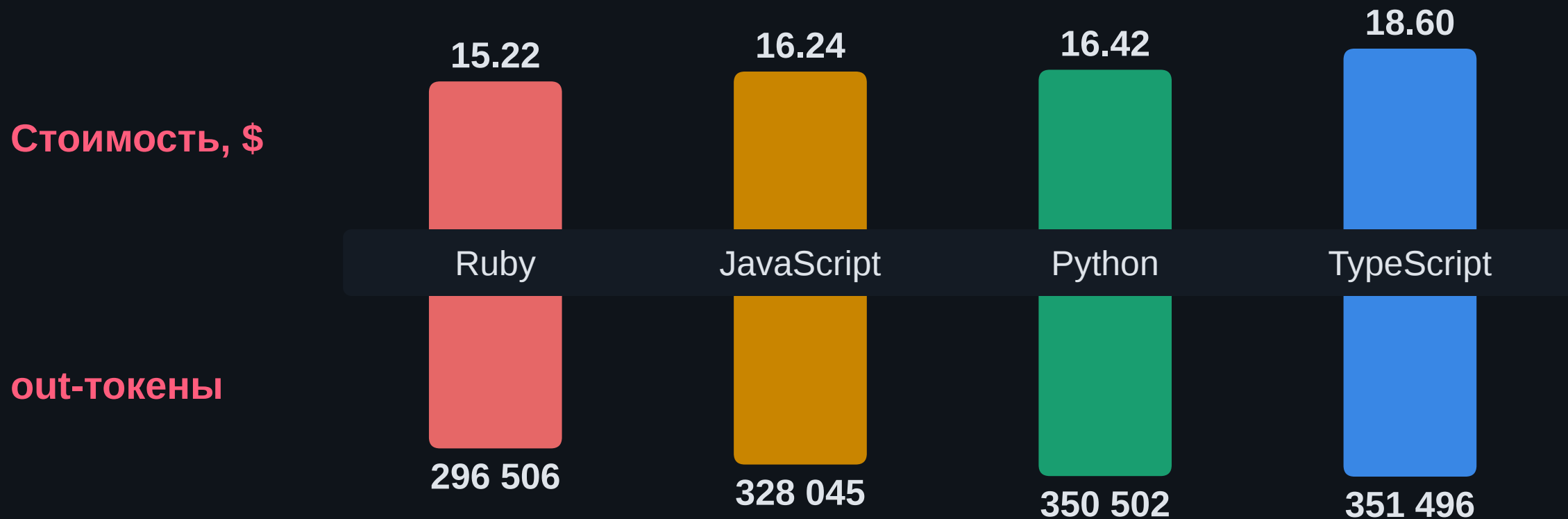
- **Ячейка** — пара «язык × тикет»: 4 языка × 11 тикетов = 44 ячейки.
- Выборка $n = 1$ на ячейку
- Типичный случай не-статистики.

Не упал никто

	Python	JavaScript	TypeScript	Ruby
Тикетов сошлось	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11
Штатные тесты	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11
Контракт зелёный	✓ 11/11	✓ 11/11	✓ 11/11	✓ 11/11

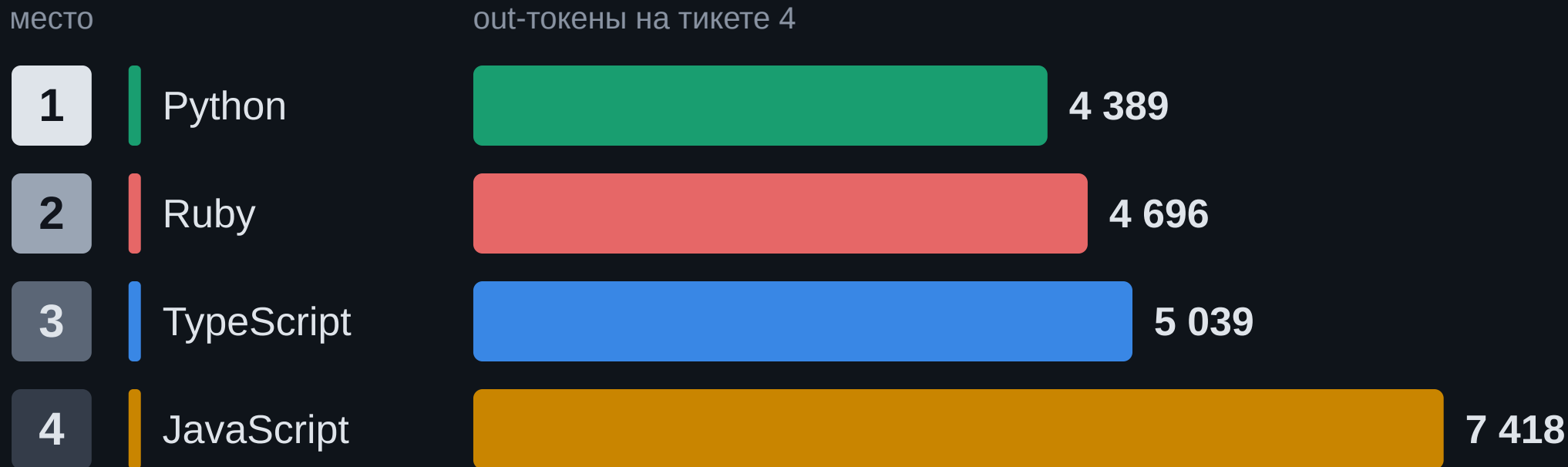
По бинарной метрике «агент справился» языкового эффекта нет — различие целиком в цене успеха.

Итого по языку



44 канонических прогона: тикеты 1–11 × 4 языка

Как считается ранг: на тикете



Тикет 4, out-токены: языки выстраиваются от меньшего значения к большему. Место 1 — потратил меньше всех, место 4 — больше всех.

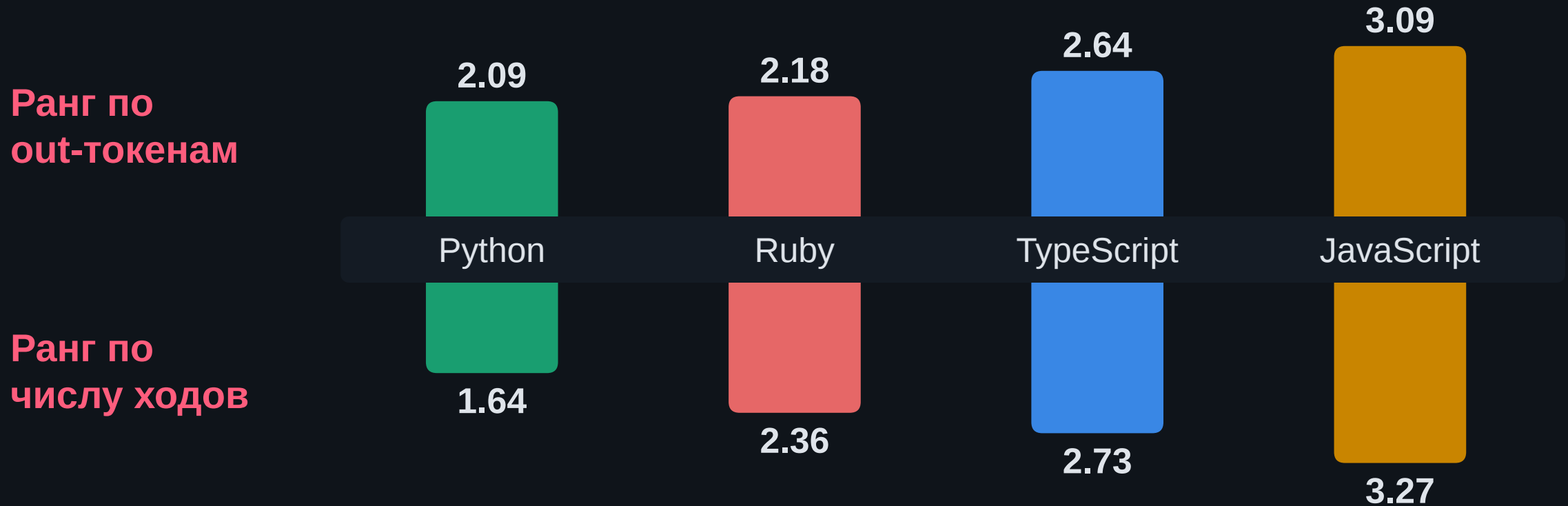
Как считается ранг: в сумме

Тикет	1	2	3	4	5	6	7	8	9	10	11	сумма ÷ 11 = ранг
Python	4	1	2	1	3	1	2	3	1	3	2	$23 \div 11 = 2.09$
Ruby	3	2	4	2	1	4	3	1	2	1	1	$24 \div 11 = 2.18$
TypeScript	2	4	1	3	4	3	1	2	3	2	4	$29 \div 11 = 2.64$
JavaScript	1	3	3	4	2	2	4	4	4	4	3	$34 \div 11 = 3.09$

тикет 4 — прошлый слайд

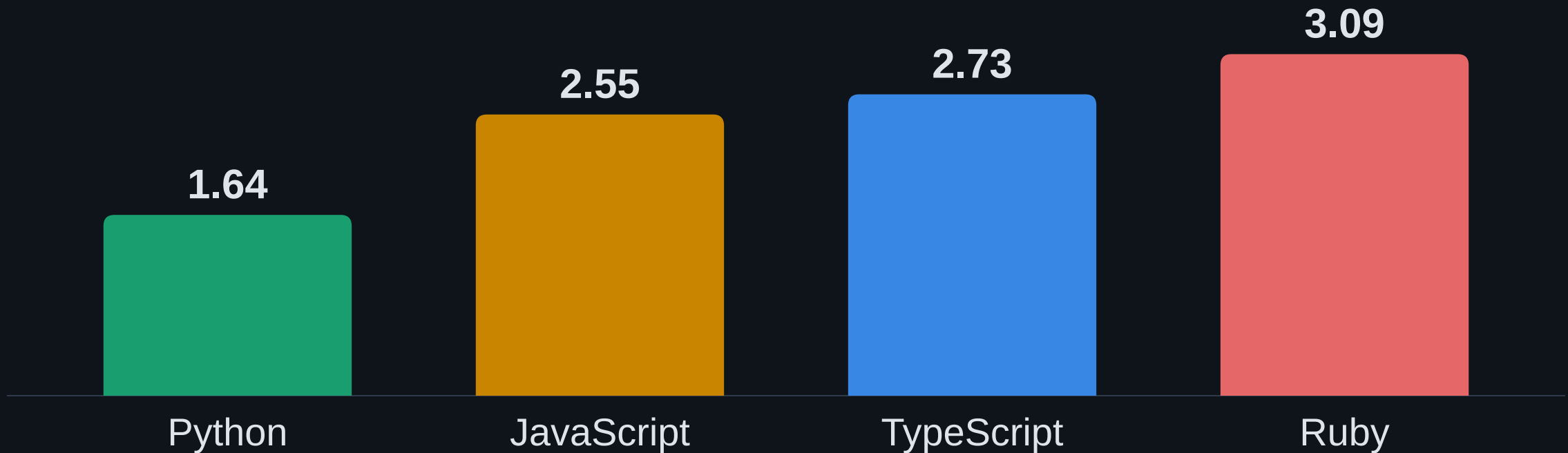
Так — на каждом из 11 тикетов. Ранг — среднее место (сумма ÷ 11); эти четыре числа — столбцы следующего слайда.
Так же считаются ранги по числу ходов и по времени.

Ранги по тикетам



Ранг 1 — дешевле/меньше всех на тикете, 4 — дороже/больше всех; среднее по 11 тикетам, снимает разброс размера тикетов. Сортировка от лучшего к худшему.

Ранги по времени



Ранг 1 — быстрее всех на тикете, 4 — медленнее всех; среднее по 11 тикетам, время — весь прогон целиком.
Сортировка от лучшего к худшему.

Ruby: разбор полётов

- Тезис «Модели знают Ruby хуже всего» **не воспроизвёлся** — под единой методикой Ruby самый дешёвый по деньгам и out-токенам
- «Какое время и какие деньги»: Ruby **не дороже**, чем {Python ≈ JavaScript}, не говоря уже о TypeScript
- Единственный Ruby-дробук: медленный «по календарю», но **не по времени модели** — разницу даёт работа вокруг модели: ~21 мин против ~7 у Python

Ruby: почему так медленно?

21.4 мин

прочее 6.8

тесты 14.6

Ruby

7.0 мин

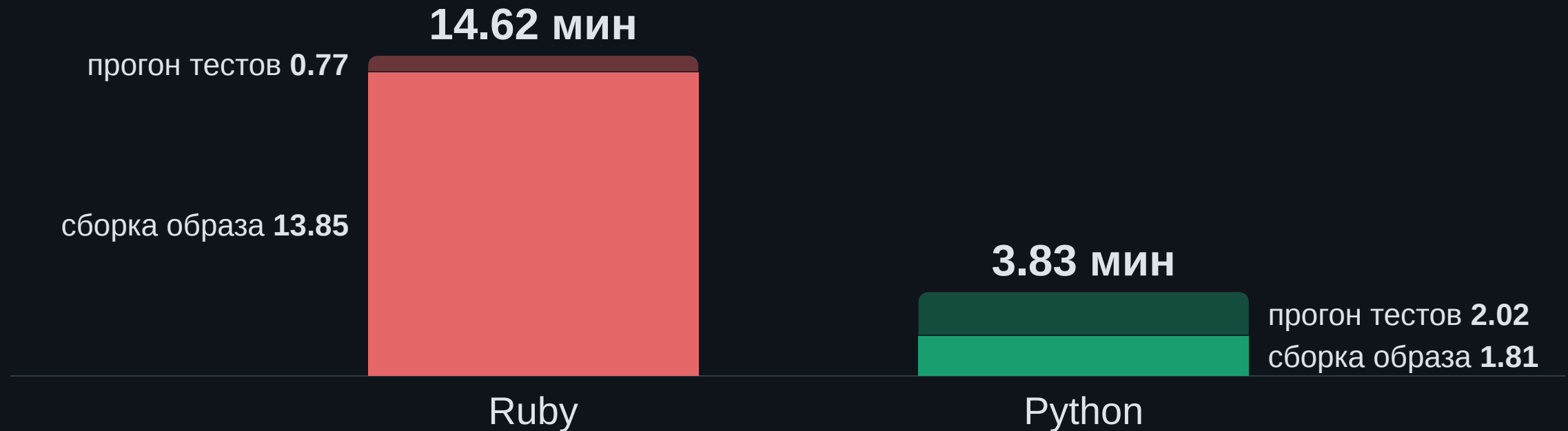
прочее 3.2

тесты 3.8

Python

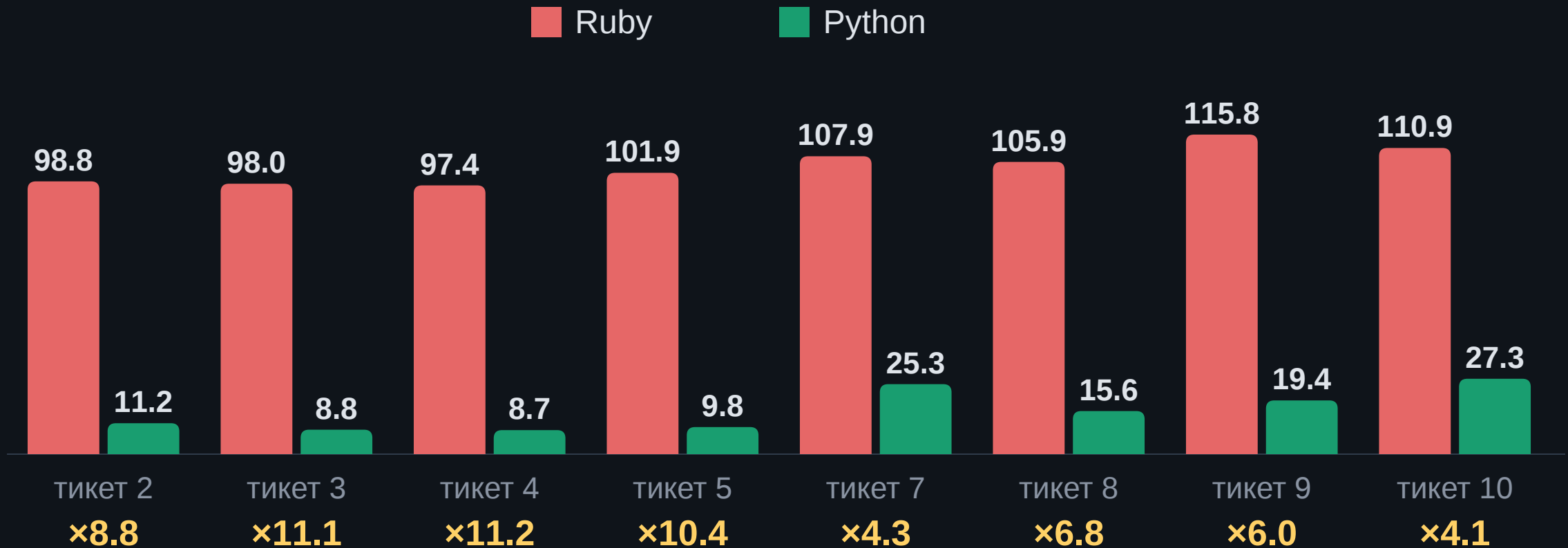
Минуты work-времени, сумма по 11 каноническим тикетам

Ruby: что внутри «тестов»



Минуты в вызовах `run-tests` — сегмент «тесты» с прошлого слайда, сумма по 11 каноническим тикетам. Сборка образа — по логу BuildKit внутри вызова; пересборка — следствие выбора образа агентом, см. ниже.

Улика: один и тот же ВЫЗОВ



Первый `./run-tests` в одном и том же тикете, секунды; ×N — во сколько раз Ruby дольше. У Ruby ~95 % вызова — сборка образа по логу BuildKit, не тесты. Тикеты 1, 6, 11 не показаны: там сборка у Ruby шла не через `run-tests`.

Ruby: причина — выбор образа, не язык

Ruby: FROM ruby:3.3

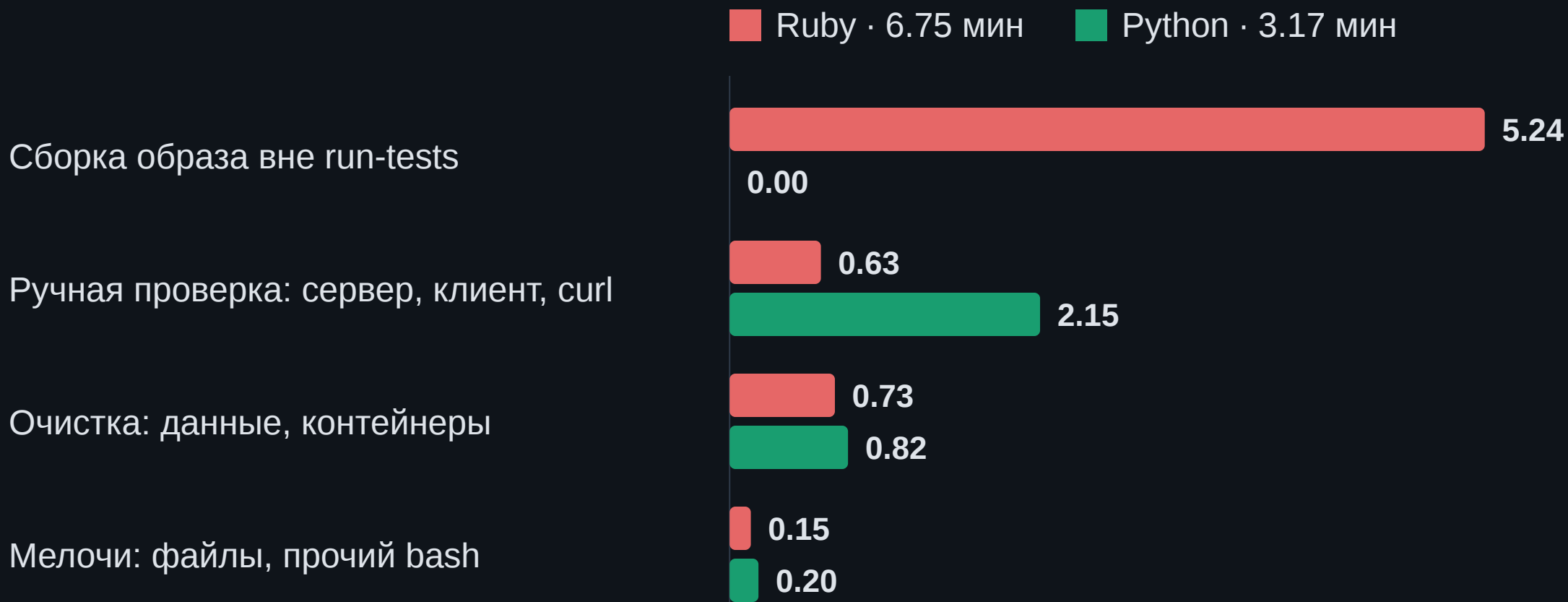
- образа нет в харнесе — заранее загружен только `ruby:3.3-slim`
- скачивание $\approx 75\text{--}90$ с
- `bundle install` $\approx 17\text{--}27$ с

Python: FROM python:3.12-slim

- образ заранее загружен в харнесс
- `pip install` ≈ 9 с

- **Образ выбрал сам агент:** FROM ruby:3.3 на тикете 1, дальше все 11 тикетов его унаследовали; до 4 сентября тот же агент писал `ruby:3.3-slim`
- `run-tests` у обоих — `docker compose run tests`: образ собирается в первом вызове, демон пуст на старте прогона
- Спецификация об образах молчит — агенту о заранее загруженных не сообщали

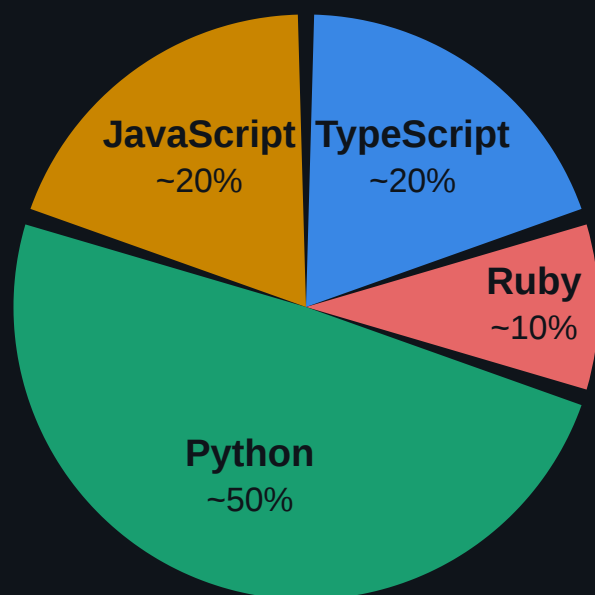
Ruby: что тормозило кроме тестов?



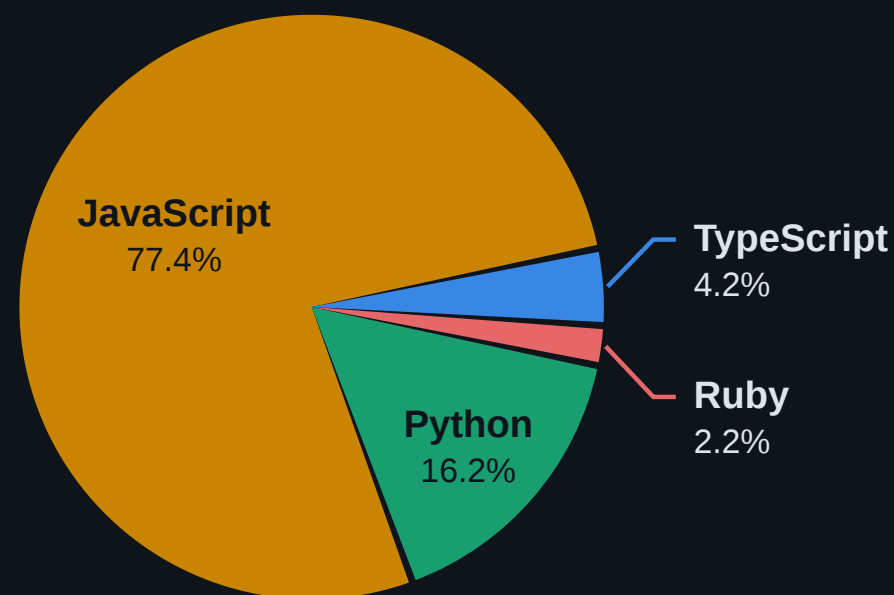
Минуты work-времени вне вызовов `run-tests`, сумма по 11 каноническим тикетам. Сборка образа вне `run-tests` — та же холодная сборка: вместе с 13.85 мин внутри вызовов это 19.09 мин, около 90 % work-времени Ruby.

Вторая аномалия: TypeScript

Ожидание

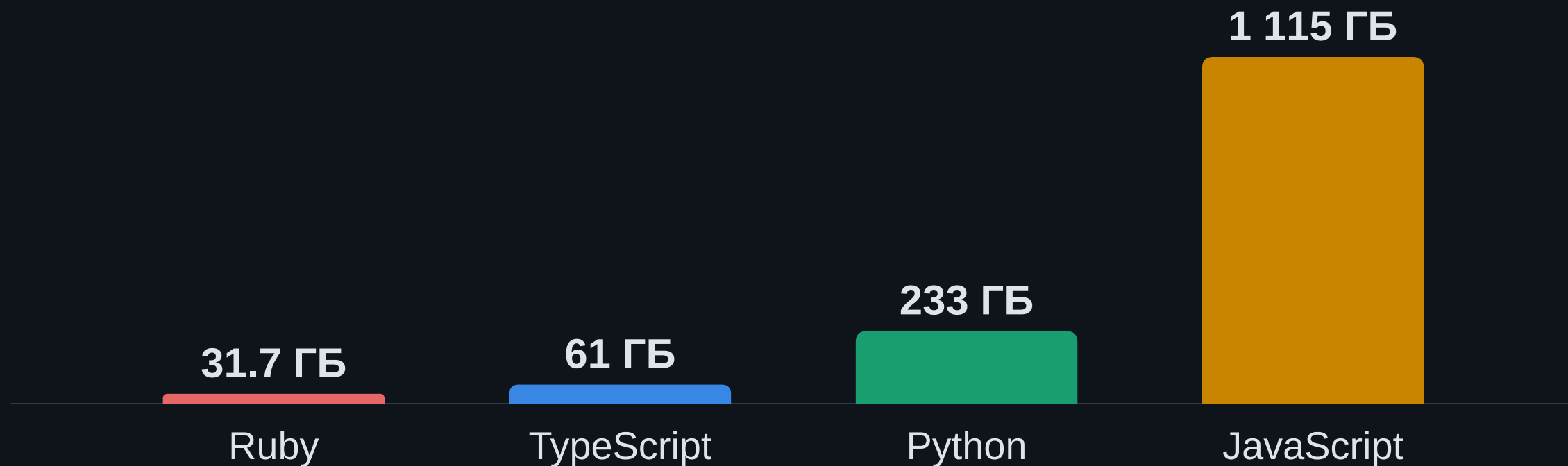


Реальность



Реальное — по The Stack v2 (BigCode), открытому корпусу исходного кода. Доли — среди этих четырёх языков. Состав обучающих данных Claude не раскрыт; открытый корпус — лучшая публичная оценка пропорций языков. [arXiv:2402.19173](#), табл. 1.

Реальный объём кода в корпусе



Ruby — вдвое меньше TypeScript и в 35 раз меньше JavaScript.

The Stack v2 (BigCode), открытый корпус исходного кода: объём в ГБ; копии одного и того же кода (почти одинаковые файлы) посчитаны один раз. Состав обучающих данных Claude не раскрыт; открытый корпус — лучшая публичная оценка пропорций языков.

[arXiv:2402.19173](https://arxiv.org/abs/2402.19173), табл. 1.

TypeScript: объяснительная записка

- TypeScript дороже всех — согласуется с тем, что его корпус в разы тоньше
- Гипотеза объединяла JS и TS в один «массовый» механизм — данные их разводят

Обзор на обзор на эксперимент

Current status

- Готов стенд
- Готов счетный аппарат
- Есть **пробные** данные

Осталось незакрытым:

- `n = 1` → нужен полный прогон (~100 тикетов на язык)
- Цена и скорость — косвенный прокси, **не анализ качества**

В заключение

Что мешает объективной оценке

- Разные проекты на разных языках не эквивалентны по масштабу
- **Локальные утечки**: модель может находить и использовать накопленный опыт в локально доступных проектах
- Разные ассистенты и инструменты — разное поведение, не связанное с языком

Практический выхлоп

- Буксует агент на Ruby — прежде чем винить язык, проверьте инструменты
- «Довольный сосед с Python» — возможно, меряет не язык, а свой стенд
- Единственный барьер, который агент не обходит изнутри, — внешняя изоляция уровня ядра ОС
- При исследовании полагайтесь на корректную методику, а не ожидаемые данные

Спасибо

Для творческих фидбеков



Обратная связь по докладу



Слайды и материалы доклада